

## *Auto-Scaling Concepts*

1. Set up a free-tier AWS account (or use an AWS Cloud sandbox) and launch a basic EC2 instance that serves a static HTML page. Document the steps you took and the public URL of your instance.

### ANS :

1. Log in to the **AWS Management Console**.
  2. Open the **EC2 Dashboard**.
  3. Click **Launch Instance**.
  4. Enter an instance name (e.g., **WebServer-ASG**).
  5. Select **Amazon Linux 2 (Free Tier Eligible)**.
  6. Choose **t2.micro** as the instance type.
  7. Create or select an existing key pair.
  8. Configure the Security Group to allow:
    - SSH (Port 22)
    - HTTP (Port 80)
  9. Launch the instance.  
**Install Apache and Create a Static Web Page**
  10. Connect to the instance using SSH:
  11. Create a simple HTML page:
    - Open the instance's **Public IP** in your browser to verify the page is accessible.
  12. **Public URL:**
    - `http://<EC2-Public-IP>`
- 
2. Create an Auto Scaling Group in AWS that maintains a minimum of 1 and a maximum of 3 EC2 instances for your web server. Set the scaling policy to add a new instance if CPU utilization goes above 60% for 5 minutes.  
  
**Hint:**  
Use the AWS Management Console's Auto Scaling Groups section and attach your EC2 launch template or configuration.

### ANS :

1. Open the **EC2 Dashboard**.
2. Create a **Launch Template** from your EC2 instance.
3. Open **Auto Scaling Groups**.
4. Click **Create Auto Scaling Group**.
5. Select the Launch Template.
6. Configure the following:
  - **Minimum Capacity: 1**
  - **Desired Capacity: 1**
  - **Maximum Capacity: 3**

7. Select your VPC and Public Subnets.
8. Attach the Auto Scaling Group to an **Application Load Balancer** (optional but recommended).
9. Create a Target Tracking Scaling Policy:
  - Metric: Average CPU Utilization
  - Target Value: 60%
  - Scale Out when CPU usage exceeds 60% for approximately 5 minutes.Finish creating the Auto Scaling Group.
3. Simulate increased traffic to your web server by using an online load testing tool (such as Loader.io or k6) and observe if the Auto Scaling Group launches additional instances when CPU usage spikes.

#### ANS :

1. Copy the Public IP or ALB DNS name.
2. Open an online load-testing tool such as **Loader.io** or run a local tool such as **k6**.
3. Send continuous HTTP requests to the web server.
4. Monitor the Auto Scaling Group and EC2 Dashboard.
  - **Observation**
    - As CPU utilization exceeds 60%, the Auto Scaling Group automatically launches additional EC2 instances (up to a maximum of 3) to handle the increased traffic.
4. After scaling up, reduce the simulated load and verify that the Auto Scaling Group scales the number of instances back down to the minimum. Take screenshots of the scaling activity history as evidence.

#### ANS :

1. Stop the load test.
2. Wait several minutes for CPU utilization to decrease.
3. Open the Auto Scaling Group dashboard.
4. Check the **Activity History**.

#### **Observation**

When CPU utilization remains low, the Auto Scaling Group terminates the extra EC2 instances and returns to the minimum capacity of **1 instance**. This reduces unnecessary resource usage and helps lower operating costs.

#### **Take screenshots of:**

- Activity History
- EC2 Instances before scaling
- EC2 Instances after scaling
- EC2 Instances after scale-in

5. Use ChatGPT or another AI tool to generate a list of 3 real-world scenarios (from apps like Zomato, Flipkart, or BookMyShow) where auto-scaling is essential, and explain in your own words why auto-scaling helps in each scenario.

ANS :

| Application | Scenario                                              | Why Auto Scaling Helps                                                                                                                                                                            |
|-------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zomato      | Meal ordering during lunch and dinner hours           | During peak meal times, thousands of users place orders simultaneously. Auto Scaling automatically adds more servers to maintain fast response times and reduces them when traffic decreases.     |
| Flipkart    | Big Billion Days or festive sales                     | Millions of customers visit the website during sales events. Auto Scaling increases the number of servers to prevent slow performance or downtime and scales them down after the sale ends.       |
| BookMyShow  | Online booking for popular movie releases or concerts | Ticket booking traffic increases rapidly when tickets go live. Auto Scaling launches additional servers to handle the high demand, ensuring users can book tickets without service interruptions. |